FINANCIAL RISK PLATFORM

WS1 | Backend Development Plan

Data Governance, Security & Platform Foundation

Workstream objective: Establish a trustworthy, secure, versioned foundation across all financial modules.

Product intent: A broad, lightweight, extensible platform foundation. Initial data volumes and module depth are limited deliberately; architecture, control points and expansion interfaces are not treated as throwaway demo code.

> **Synchronization rule: This document is paired with the opposite-side document for the same workstream. Feature IDs, phase IDs, endpoint contracts, statuses and acceptance criteria must remain aligned.**

Shared architecture: Layer 1 Universal Platform Foundation → Layer 2 Configurable Financial Domain → Layer 3 Financial Calculation Modules → Layer 4 Intelligence and Experience. Data governance and security cross-cut all layers.

1. Scope and Feature Coverage

Feature ID	Feature	Scope / implementation intent
DG-01	Canonical schema and metadata	Define schema registry, version, owner, effective date, compatibility and lifecycle.
DG-02	Identity and authorization	Enforce server-side roles/permissions and least privilege.
DG-03	Data classification and protection	Classify fields, mask sensitive values, protect secrets and prevent sensitive logging.
DG-04	Audit and provenance	Record critical access/actions, actor, timestamp, resource and correlation ID.
DG-05	Controlled model change	Review schema/mapping changes, assess downstream impact, test and approve before release.
DG-06	Environment and retention controls	Separate synthetic/test data from production; define retention, deletion and backup assumptions.

2. Phase-Wise Delivery

P0 | Foundation contract

Agree data/security assumptions, roles, classifications and architecture boundaries.

P1 | Schema and metadata registry

Versioned schemas, ownership, lifecycle and compatibility metadata.

P2 | Security and audit controls

Authentication integration, server-side authorization, audit records, masking and safe logging.

P3 | Change governance

Schema diff, compatibility checks, impact map, approval status and rollback/recovery plan.

P4 | Cross-platform enforcement

Connect other workstreams to shared identity, schema, audit and version services.

P5 | Verification

Test forbidden access, breaking schema changes, audit completeness and recovery paths.

3. Implementation Responsibilities

Backend responsibilities

• Define schema registry and immutable version records with lifecycle/status metadata.

• Implement authorization middleware and permission checks on every protected endpoint.

• Define field classification and masking policy; avoid sensitive payloads in logs.

• Write append-oriented audit events for schema changes, approvals, exports and sensitive operations as required.

• Implement schema diff, compatibility classification and dependency impact service.

• Require approval for breaking changes and preserve a recovery path; never silently reinterpret historic records.

4. Paired API / Feature Contract Matrix

Endpoint names are proposed contracts for alignment, not a claim that the APIs already exist. Confirm payload schemas in the shared OpenAPI contract before implementation.

Shared feature pair	Capability	Proposed API contract
FE-DG-01 / BE-DG-01	Schema catalogue/version compare	GET /api/v1/schemas; GET /api/v1/schemas/{schema_id}/versions; GET /api/v1/schemas/{schema_id}/diff
FE-DG-02 / BE-DG-02	Permissions	GET /api/v1/me/permissions; authorization enforced on protected APIs
FE-DG-03 / BE-DG-03	Audit timeline	GET /api/v1/audit-events?resource_id={id}
FE-DG-04 / BE-DG-04	Change review	POST /api/v1/schema-change-proposals; GET /api/v1/schema-change-proposals/{id}; POST /api/v1/schema-change-proposals/{id}/impact-analysis; POST /api/v1/schema-change-proposals/{id}/approval

5. Cross-Cutting Rules

• The canonical model and source mapping are versioned separately. Unknown source attributes are preserved or quarantined by explicit policy, not silently discarded.

• Every calculation run must bind dataset snapshot, schema/mapping version, rule-set version and calculation-engine version.

• Server-side authorization is authoritative. The frontend may hide unavailable actions but does not enforce security.

• No AI-generated code is executed. AI output is validated against schemas and policy, and consequential actions require authorized approval.

• All errors use a structured response with code, message, details and correlation_id. Sensitive payloads and secrets must not be logged.

• Prototype financial modules use synthetic/approved data, documented assumptions and test fixtures; do not claim regulatory certification or Moody's parity.

• The platform is designed for breadth, but production-grade depth is not implied for every financial domain. ALM/IRRBB, collateral, capital, live market data and regulatory feeds require their own specifications and validation.

6. Definition of Done

• Every feature ID in this document has a matching feature ID in its paired frontend/backend document and in the shared feature register.

• Each endpoint has request/response schemas, status enums, authorization expectations and error examples in OpenAPI.

• Frontend components use actual API responses or clearly labelled mocks matching the contract.

• Backend tests cover validation, permissions, version binding and domain logic appropriate to this workstream.

• The cross-workstream integration test passes for the features included in the release.

Change-control rule: Any change to endpoint, field name, enum, versioning behaviour or acceptance criteria must be reflected in both paired documents and the shared contract in the same change.